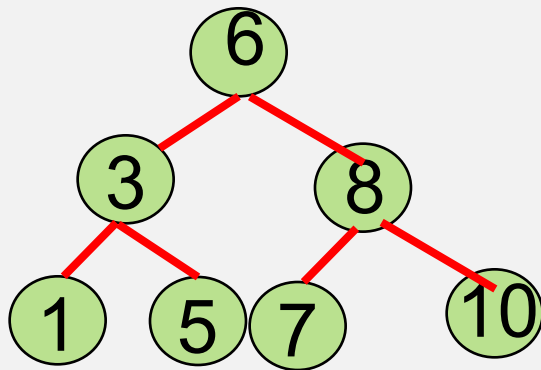




การ insert Binary Search Trees

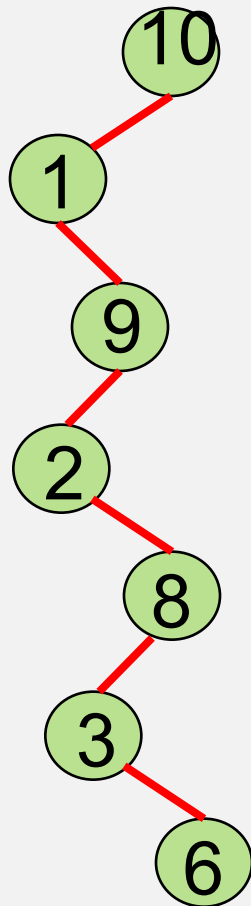
6 3 8 1 5 7 10



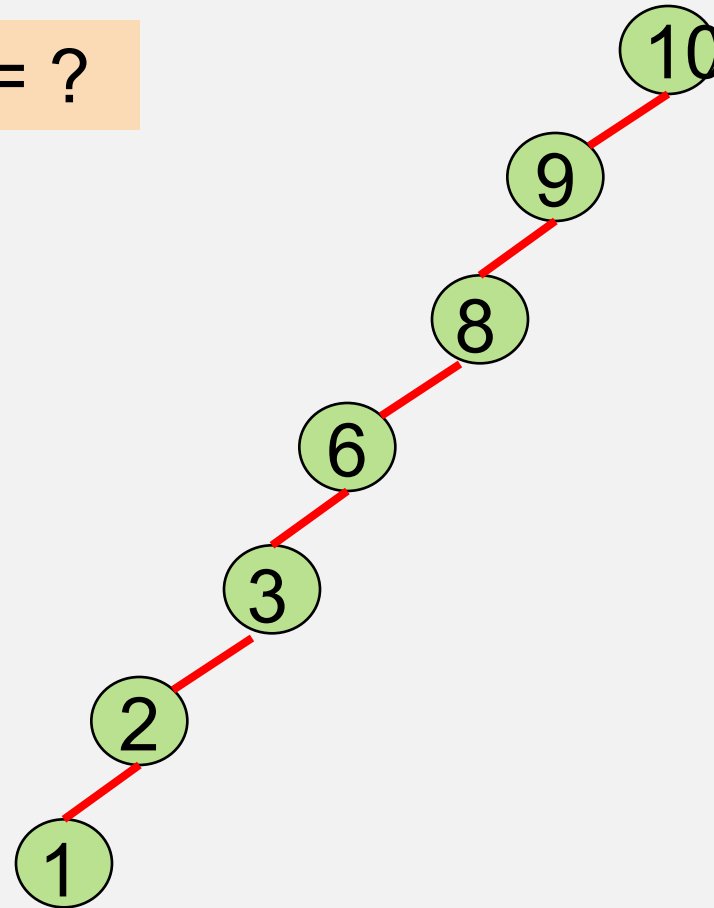


Insert : 10 1 9 2 8 3 6

Insert : 10 9 8 6 3 2 1



Big O = ?





ปัญหา Big O ของ Binary Search Trees

worse case $O(n)$

การแก้ไข

- ทำให้ Trees balances
- A binary search tree with **balance condition**, it ensures that the depth of the tree is $O(\log_n)$.
- **Balance condition** : Every node must have left and right subtree of the same height.
- ต้องทำให้หรือ Perfect Balance Trees : ทำยาก



3.4 AVL Trees

Problem : Only **perfect balance trees** of $2^k - 1$ node would satisfy this criterion.

การแก้ไข

AVL Trees

AVL : Binary Search Trees ที่มี Balance condition
(ความสูงต่างกันไม่เกิน 1)

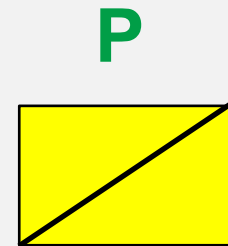
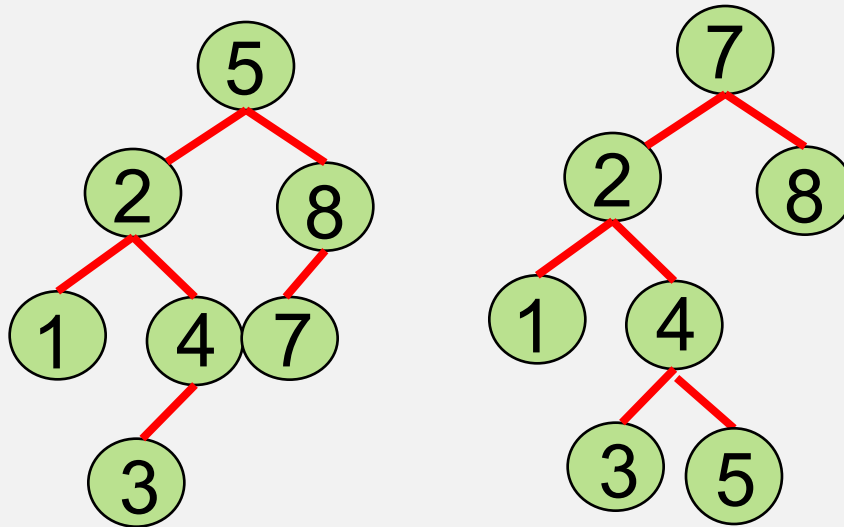
ด้วยวิธีการ

- Single Rotation
- Double Rotation



Definition : An AVL(Adelson-Velskii and Landis) tree is identical to a binary search tree, except that for every node in the tree, the height of the left and the right subtree **can differ by at most 1**.

AVL Operation All = $O(\log_N)$ except insertion



The height of an empty subtree is defined to be -1.



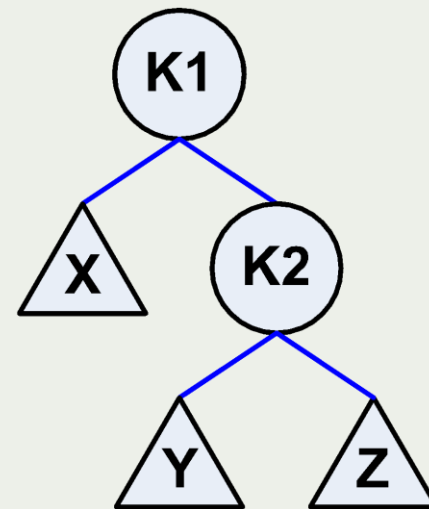
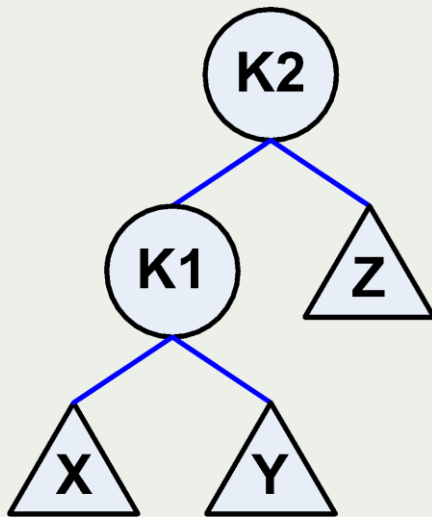
3.4.1 Insertion operation :

1. Update all the balancing information for the nodes on the path back to the root
2. The insertion a node could violate the AVL tree property, then property has to be restored before the insertion step. Called a *rotation*
 - **Single rotation**
 - **Double rotation**



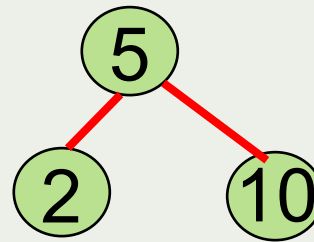
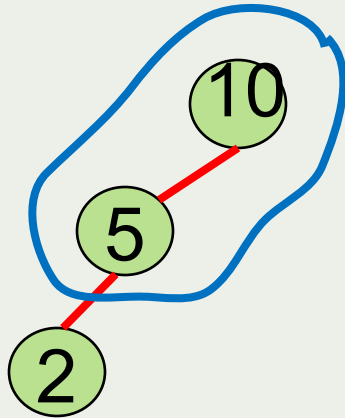
3.4.2 Single Rotation

- A rotation involves only a few pointer changes, and changes the structure of the tree while preserving tree property.



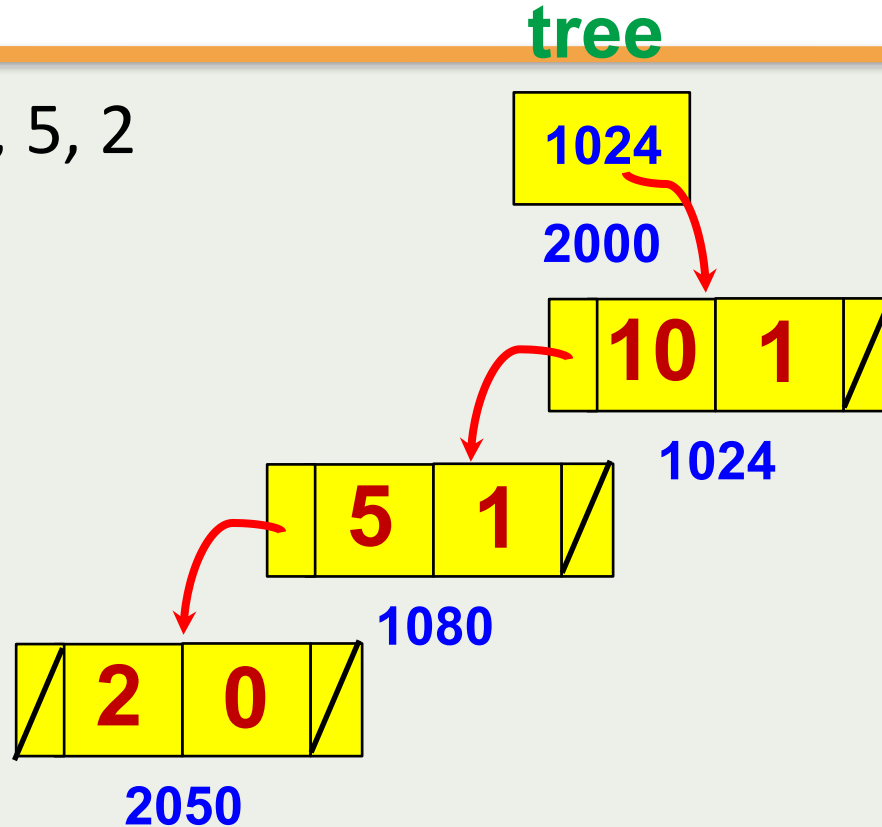
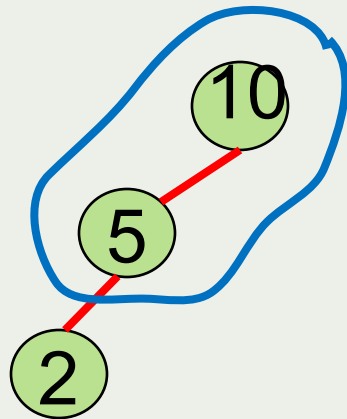


Example insert 10, 5, 2 : rotate right ตามเข็มนาฬิกา





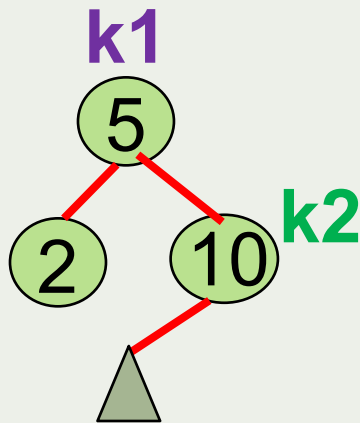
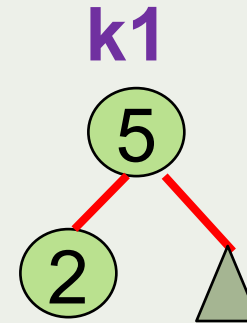
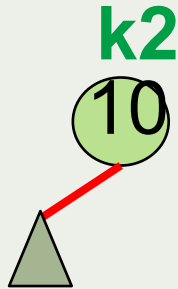
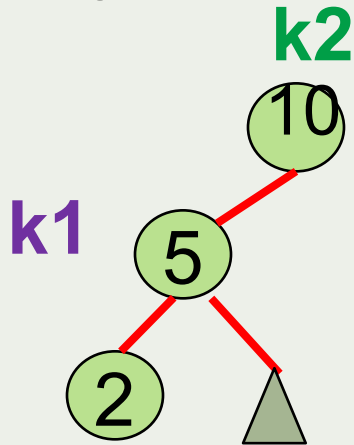
Example insert 10, 5, 2



นอกช่วงของจุดเสียสมดุลกับจุดถัดไป
struct node *srleft(struct node *k2)
struct node *srright(struct node *k2)
struct node *srright(struct node *k2)



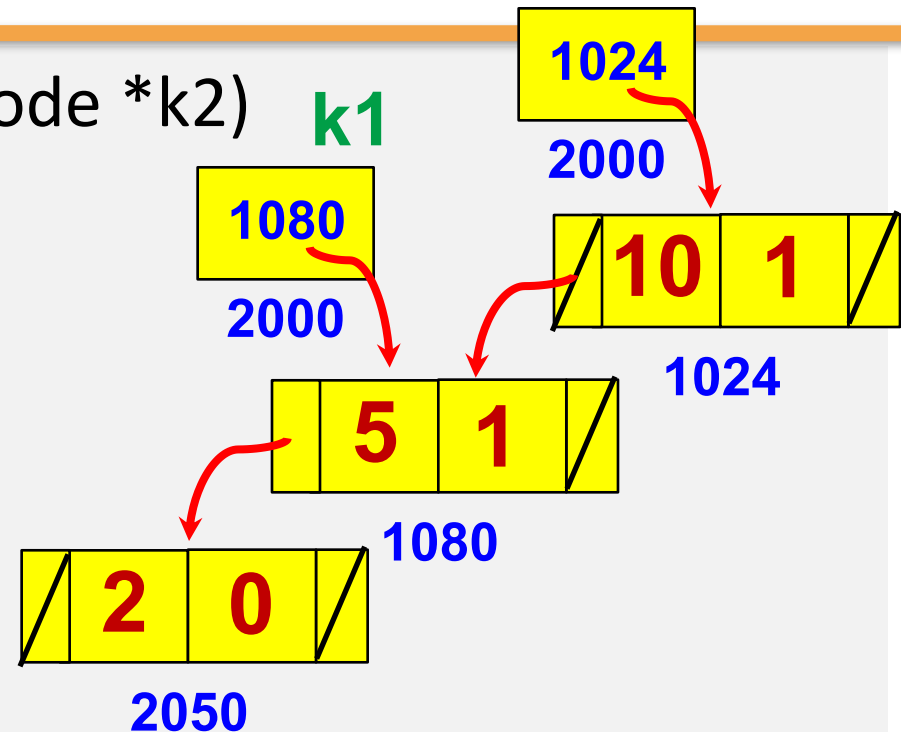
Example insert 10, 5, 2 เรียกการเอียงแบบนี้ว่า LL



```
struct node *srright (struct node *k2)
{
    struct node *k1;
    k1=k2->left;
    k2->left = k1->right;
    k1->right = k2;
}
```



```
struct node *srright(struct node *k2)
{
    struct node *k1;
    k1=k2->left;
    k2->left = k1->right;
    k1->right = k2;
}
```



Example insert 1, 2, 3, 4, 5, 6, 7

นอกช่วง

Example insert 15, 14

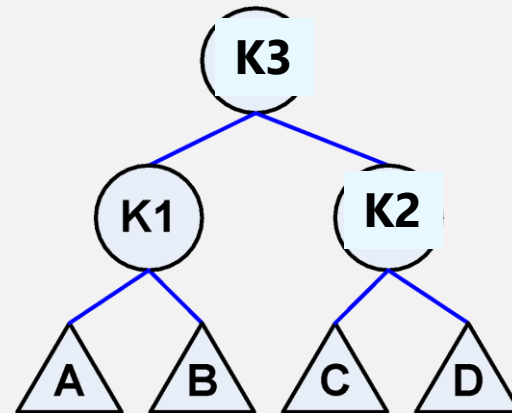
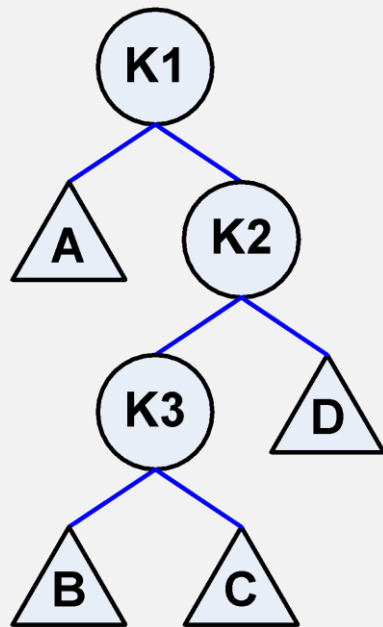
insert 13, 12, 11, 10, 9, 8, 8.5

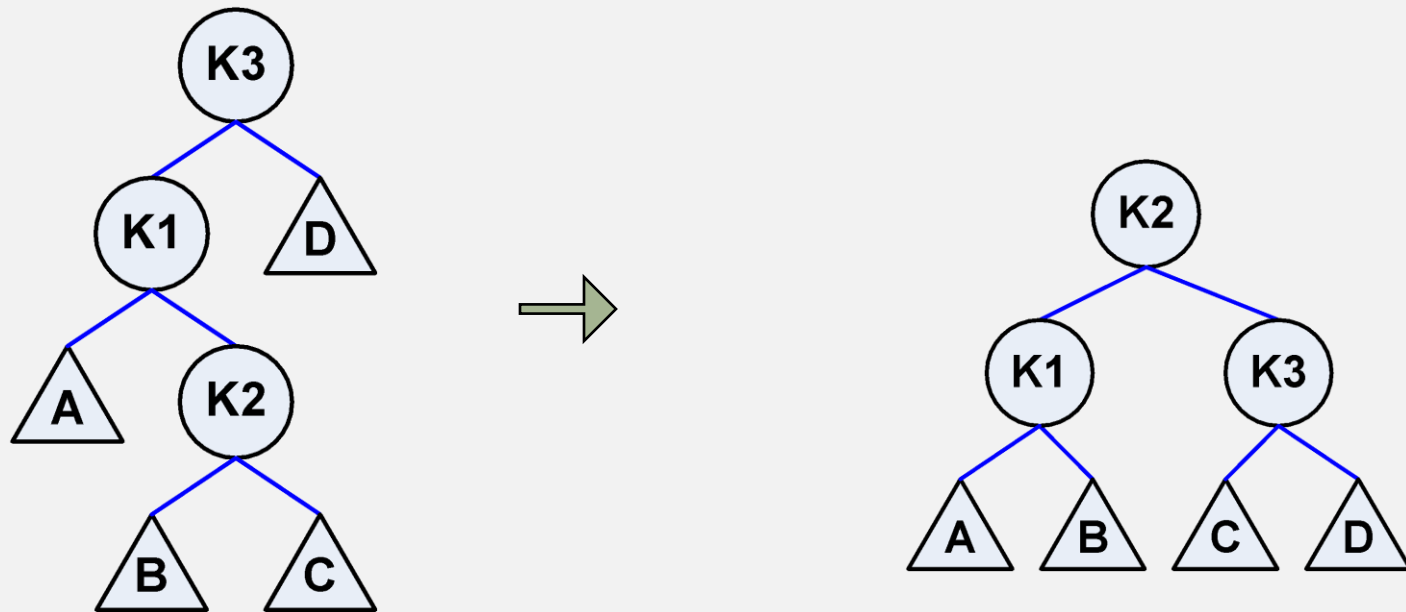
นอกช่วง



3.4.3 Double rotation

Problem : Single rotation has not fixed the height imbalance by a node inserted into the tree containing the middle elements.





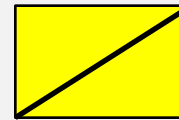


```
struct node
{
    int value;
    int height;
    struct node *left;
    struct node *right;
};
```

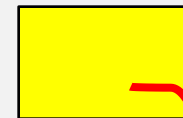
```
1. int fheight( struct node *P)
2. { if ( P==NULL)
3.     return -1;
4.     else
5.     return P->height;
6. }
```



P



P



1024



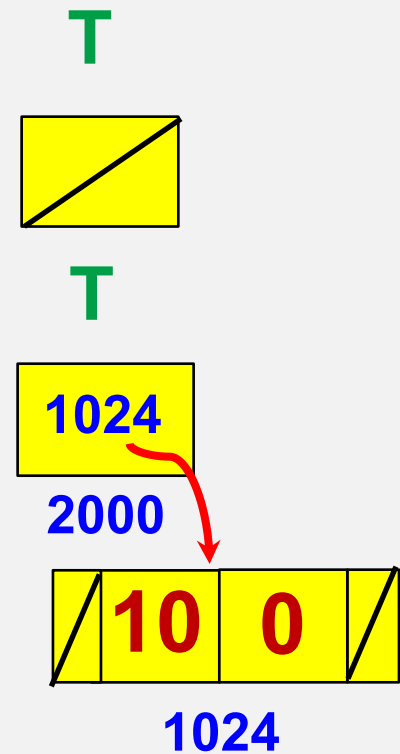
Code Insert

```
1 ถ้า tree == NULL
2   สร้าง node ใหม่ ใส่ค่า
3 else
4   ถ้า x น้อยกว่า tree->value ให้ recursive ลงไปทางซ้าย
5   tree->left = insert (tree->left)
6   ถ้าความสูงของทรีทางซ้ายและทางขวาต่างกันเกิน 2
7     ถ้า x น้อยกว่า tree->left->value
8       single rotation (อยู่นอกช่วง)
9     else
10      double rotation (อยู่ในช่วง)
11 update ความสูง
12 return tree
```

เมื่อ insert แล้ว return
กลับมาเช็คความสูง



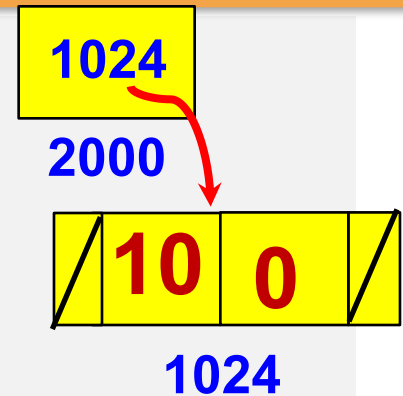
```
struct node *insert(int x, struct node *T)
1{   if(T == NULL)
2   {   T=new struct node;
3       T->value=x;
4       T->left=T->right=NULL;
5       T->height=0;
6   }
   else
```





```
8 if(x < T->value )
9     { T->left = insert(x,T->left);
10         if(fheight(T->left) - fheight(T->right) == 2)
11             if( x < T->left->value)
12                 T=srright(T);
13             else
14                 T=dLR(T);
15     }
16 else
17 ถึง 29 if(x > T->value )
18     { ข้างขวาบ้าง ให้เขียนเอง ... }
```

```
30 T->height =  $\max(-1, -1) = -1$  +1;
31 return T;
}
```

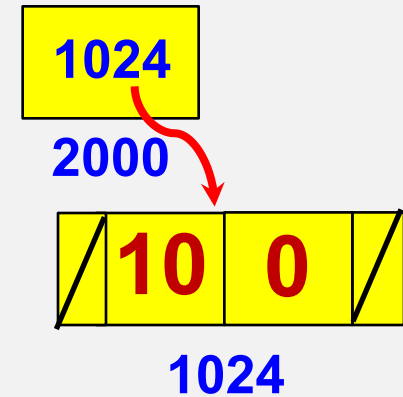




```
struct node *insert(int x, struct node *T)
1{  if(T == NULL)
2    {  T=new struct node;
3        T->value=x;
4        T->left=T->right=NULL;
5        T->height=0;
        }
7  else
```



tree



ถ้า x น้อยกว่า tree->value
ให้ recursive ลงไปทางซ้าย

– Tree

tree

21

5, NULL

5

```
8 if(x < T->value )
9   { T->left = insert(x,T->left);
10     if(fheight(T->left) - fheight(T->right) == 2)
11       if( x < T->left->value)
12         T=srright(T);
13       else
14         T=dLR(T);
15   }
```

16else

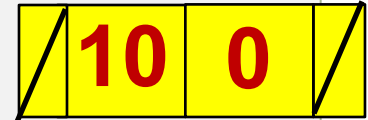
```
17ถึง 29 if(x > T->value )
    { ข้างขวาบ้าง ให้เขียนเอง ... }
```

```
30 T->height = max(fheight(T->left), fheight(T->right)) + 1;
```

```
31 return T;
}
```

1024

2000



1024

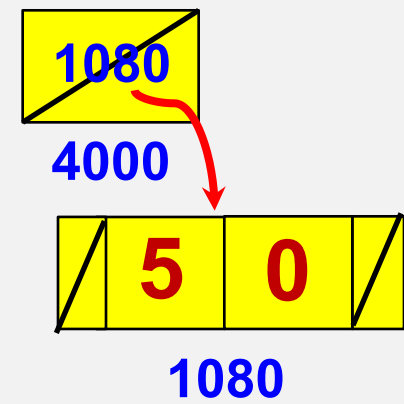


Recursive 1

5, NULL

```
struct node *insert(int x, struct node *T)
1{  if(T == NULL)
2    {  T=new struct node;
3        T->value=x;
4        T->left=T->right=NULL;
5        T->height=0;
        }
7  else
    ....
```

tree



$$\text{max}(-1, -1) = -1 \quad +1$$

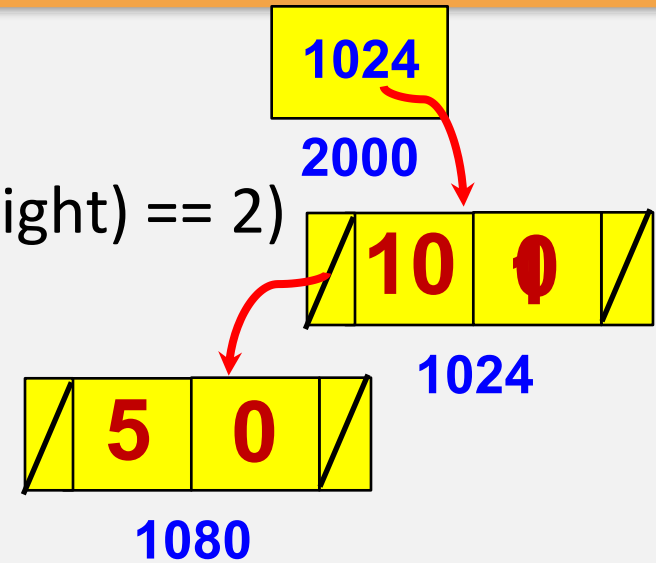
```
30  T->height = max(fheight(T->left), fheight(T->right)) + 1;
31  return T;
    }
```

1080



```
8 if(x < T->value )
9   { T->left = insert(x,T->left);
10     if(fheight(T->left) - fheight(T->right) == 2)
11       if( x < T->left->value)
12         T=srright(T);
13       else
14         T=dLR(T);
15   }
16ถึง 29 if(x > T->value )
17   { ข้างขวาบ้าง ให้เขียนเอง ... }
30 T->height = max(fheight(T->left), fheight(T->right)) + 1;
31 return T;
}
```

1080

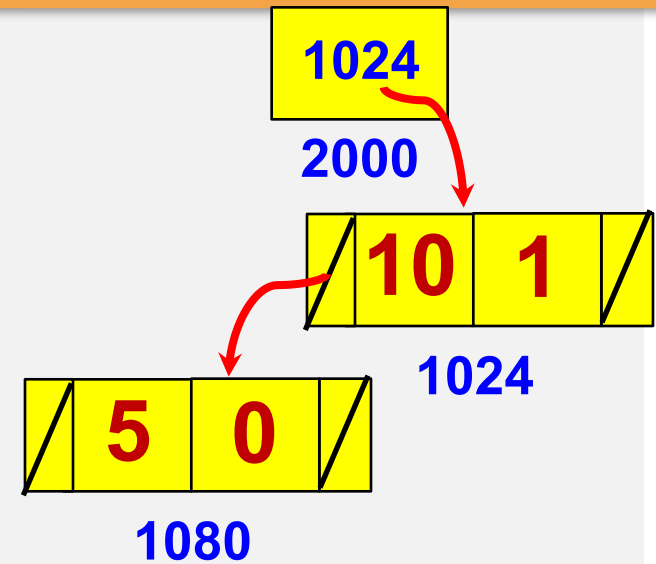




```
main()  
{
```

```
.....
```

```
tree=insert(2,tree);
```



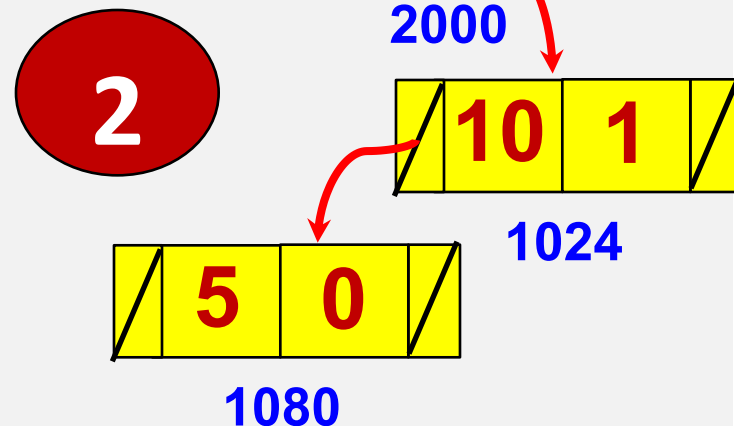


จาก main

2, 1024

```
struct node *insert(int x, struct node *T)
```

```
1{   if(T == NULL)
2   {   T=new struct node;
3       T->value=x;
4       T->left=T->right=NULL;
5       T->height=0;
        }
7   else
        ....
```



ถ้า x น้อยกว่า tree->value
ให้ recursive ลงไปทางซ้าย

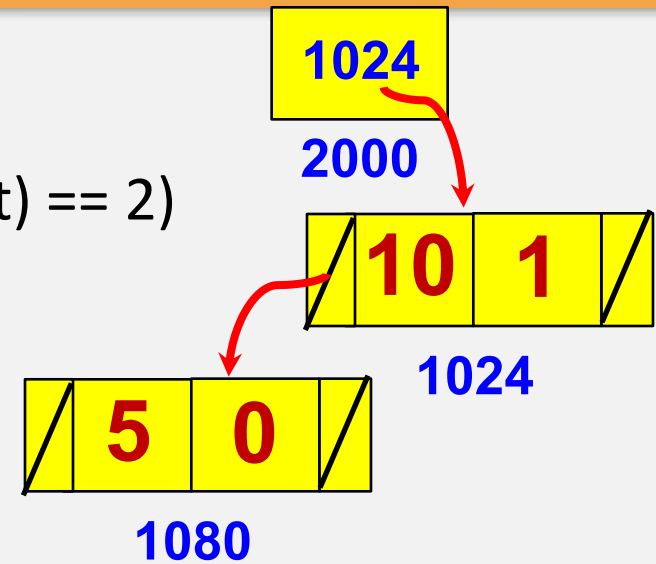
Tree

tree

26

```
8 if(x < T->value )
9   { T->left = insert(x,T->left);
10     if(fheight(T->left) - fheight(T->right) == 2)
11       if( x < T->left->value)
12         T=srright(T);
13       else
14         T=dLR(T);
15   }
16ถึง 29 if(x > T->value )
17   { ข้างขวาบ้าง ให้เขียนเอง ... }
30 T->height = max(fheight(T->left), fheight(T->right)) + 1;
31 return T;
}
```

2, 1080



(A) T = 1024 ค้าง 9 หน้า 26

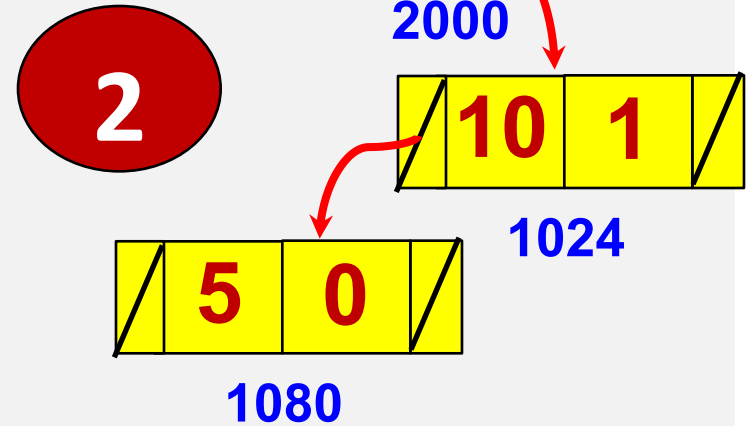


Recursive 1

2, 1080

```
struct node *insert(int x, struct node *T)
```

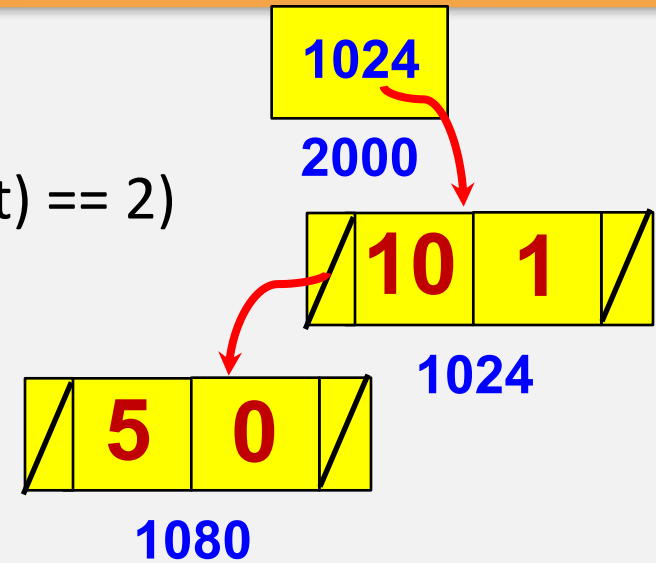
```
1{   if(T == NULL)
2   {   T=new struct node;
3       T->value=x;
4       T->left=T->right=NULL;
5       T->height=0;
        }
7   else
        ....
```





tree

```
8 if(x < T->value )
9     { T->left = insert(x,T->left);
10         if(fheight(T->left) - fheight(T->right) == 2)
11             if( x < T->left->value)
12                 T=srright(T);
13             else
14                 T=dLR(T);
15     }
16ถึง 29 if(x > T->value )
17     { ข้างขวาบ้าง ให้เขียนเอง ... }
30 T->height = max(fheight(T->left), fheight(T->right)) + 1;
31 return T;
}
```

2, NULL**(B) T = 1080 ค้างบรรทัด 9 หน้า 28**



Recursive 2

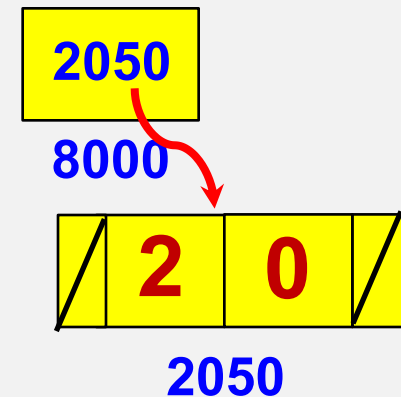
2, NULL

```
struct node *insert(int x, struct node *T)
```

```
1{   if(T == NULL)
2   {   T=new struct node;
3       T->value=x;
4       T->left=T->right=NULL;
5       T->height=0;
        }
7   else
        ....
```



tree



```
30   T->height =  $\max(-1, -1) = -1$   $+1$ 
31   return T;
}
```

กลับไปที

(B) T = 1080 ค้างบรรทัด 9 หน้า 28

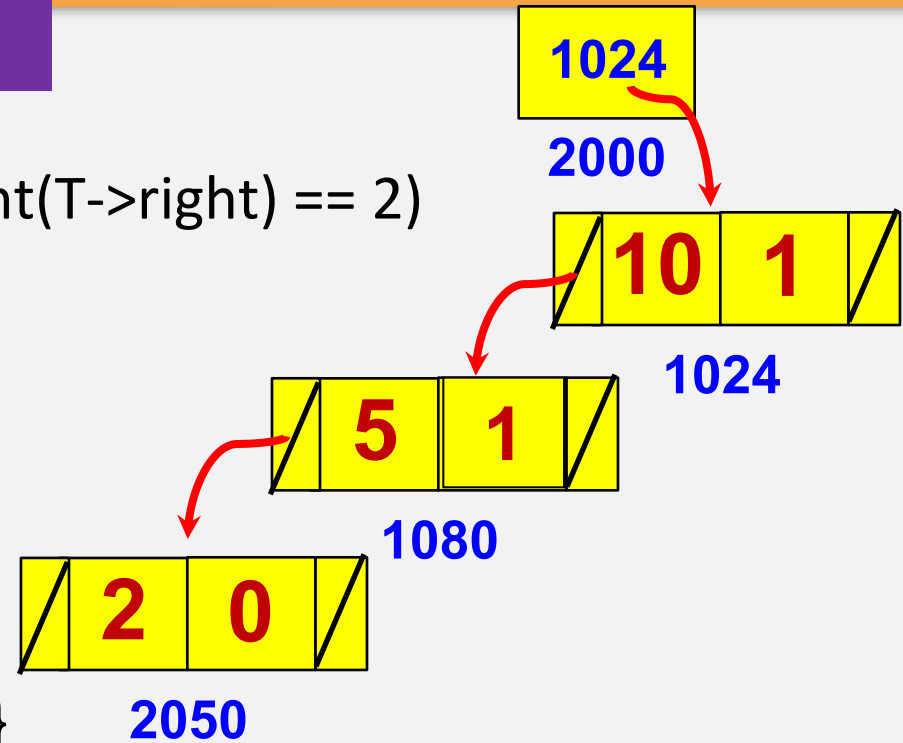
(B) T = 1080 ค้างบรรทัด 9

tree

30

```
8 if(x < T->value )
9     { T->left = insert(x,T->left);
10         if(fheight(T->left) - fheight(T->right) == 2)
11             if( x < T->left->value)
12                 T=srright(T);
13             else
14                 T=dLR(T);
15     }
16ถึง 29 if(x > T->value )
17     { ช้างขวาบ้าง ให้เขียนเอง ... }
30 T->height = max(fheight(T->left), fheight(T->right)) + 1;
31 return T;
}
```

2050





(A) ค้างบรรทัด 9 1024

– Tree

tree

31

8 if(x < T->value)

1080

9 { T->left = insert(x,T->left);

10 if(fheight(T->left) - fheight(T->right) == 2)

11 if(x < T->left->value)

12 T=srright(T);

13 else

14 T=dLR(T);

15 }

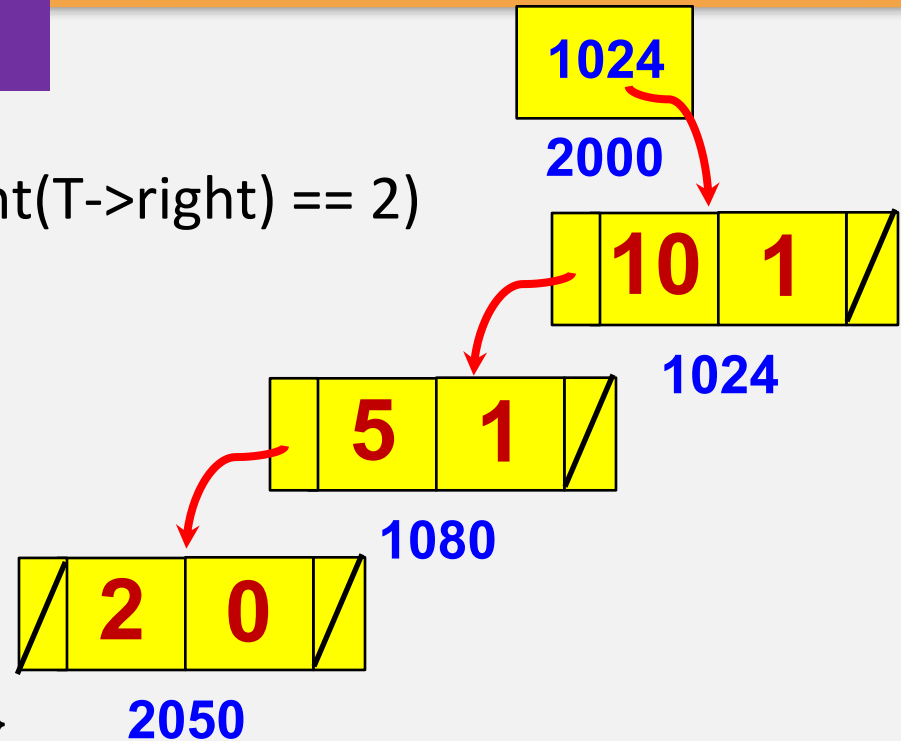
16ถึง 29 if(x > T->value)

{ ช้างขว้าง ให้เขียนเอง ... }

30 T->height = max(fheight(T->left), fheight(T->right)) + 1;

31 return T;

}



```

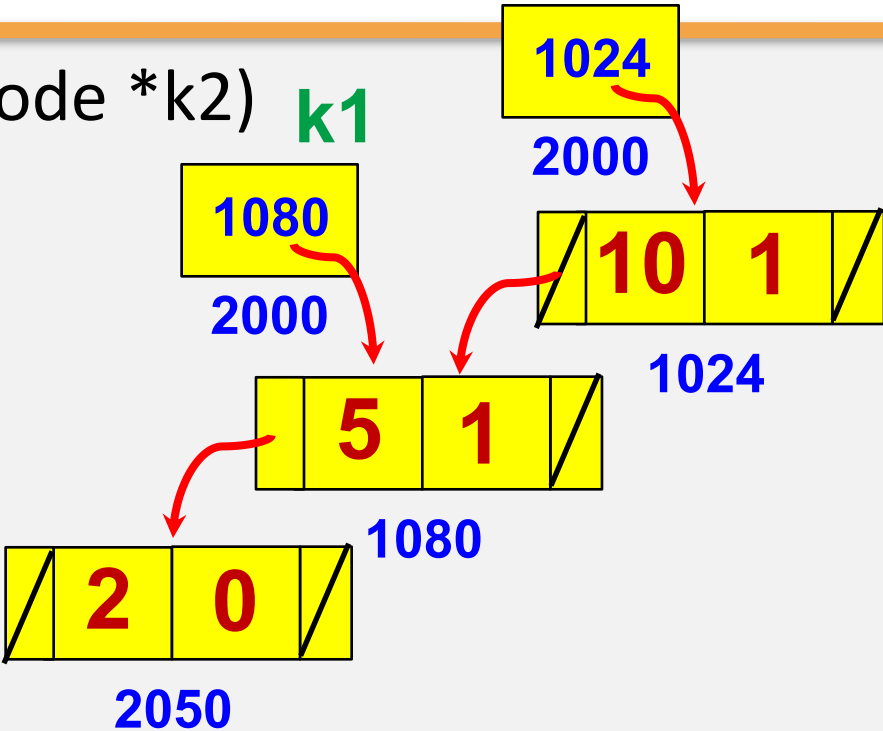
struct node *srright(struct node *k2)

```

```

{
    struct node *k1;
    k1=k2->left;
    k2->left = k1->right;
    k1->right = k2;

```



```

    k2->height = max(fheight(k2->left), fheight(k2->right)) + 1;
    k1->height = max(fheight(k1->left), k2->height) + 1;
    return k1;
}

```



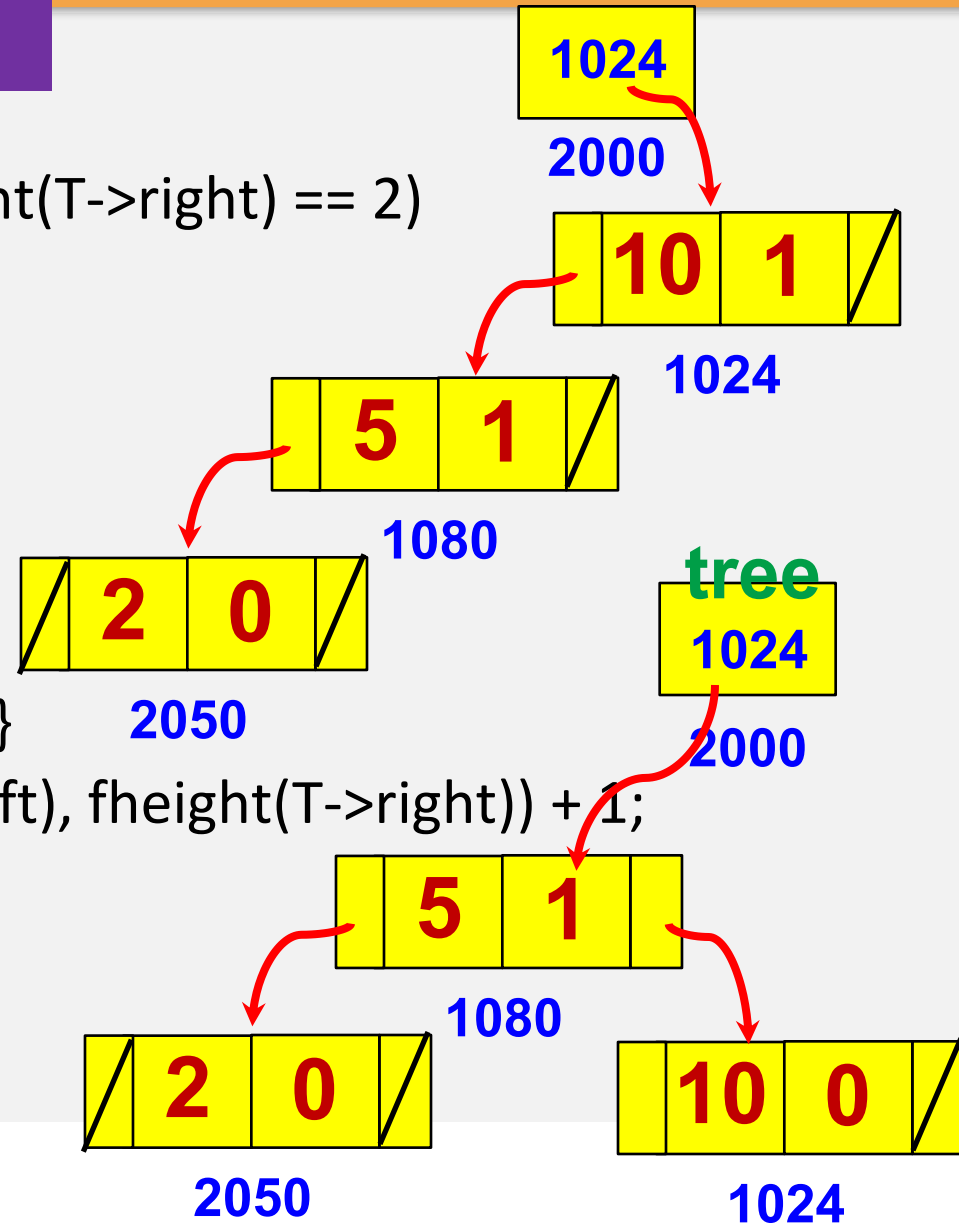
A ค้างบรรทัด 9 1024 le3 - Tree

33

tree

```
8 if(x < T->value )
9   { T->left = insert(x,T->left);
10     if(fheight(T->left) - fheight(T->right) == 2)
11       if( x < T->left->value)
12         T=srright (T);
13     else
14       T=dLR (T);
15   }
16ถึง 29 if(x > T->value )
17   { ช้างขวาบ้าง ให้เขียนเอง ... }
30 T->height = max(fheight(T->left), fheight(T->right)) + 1;
31 return T;
}
```

1080





```
struct node *drLR(struct node *k3)
{
    k3->left = srleft (k3->left);
    return srright (k3);
}
```



ทบทวน

Perfect balance trees : Binary search trees ที่มี balanced condition ต้องมี node $2^k - 1$



AVL Trees : a binary search tree, except that for every node in the tree, the height of the left and the right subtree can differ by at most 1.

- single rotation

 - 3, 2, 1

 - 1, 2, 3

- double rotation

 - 3, 1, 2

 - 1, 3, 2



3.5 Splay Trees

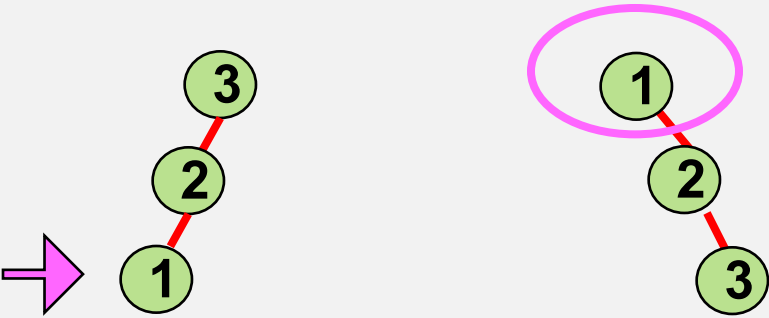
- มี binary search tree อยู่แล้ว แต่ไม่ balance
- $O(N)$ worst-case time per operation for binary search trees.

แก้ปัญหารีที่มีอยู่แล้วแต่ไม่ balance ด้วยเทคนิค splay trees

- กรณีมีการใช้งาน(accessed) ข้อมูลใน trees เช่น การค้นหาข้อมูล จะทำการค้นหาและนำข้อมูลที่ค้นหาได้แล้วนั้นกลับขึ้นมาเป็น root
- After a node is accessed, it is pushed to the root

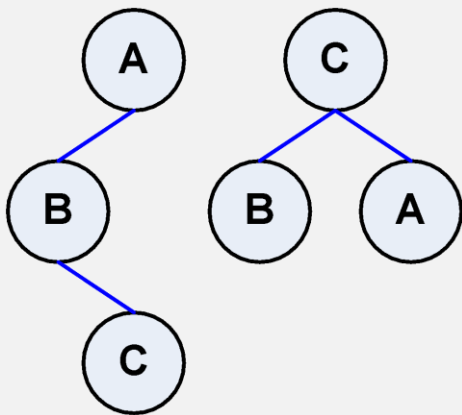
3.5 Splay Trees

- $O(N)$ worst-case time per operation for binary search trees.
- After a node is accessed, it is pushed to the root

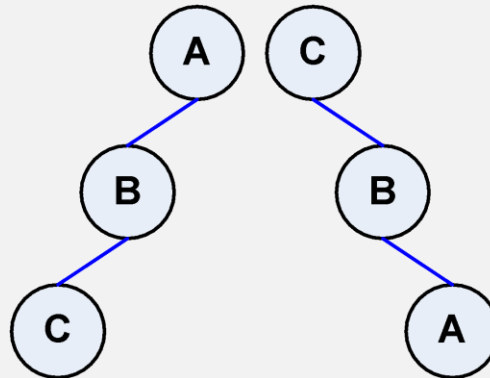


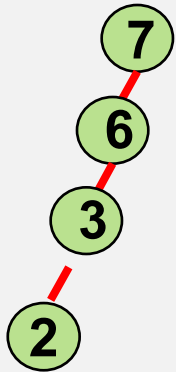
Rotation idea (วิธีการนำ node ที่ถูก accessed ขึ้นมาเป็น root) เรียกว่า Transform

1. zig-zag

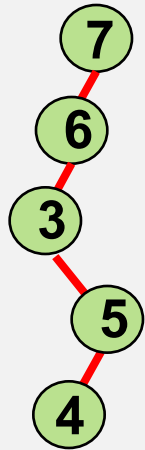


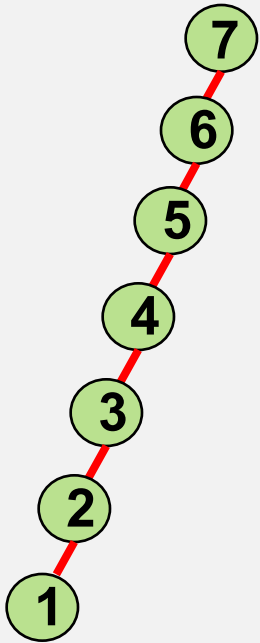
2. zig-zig





ถ้า zig-zag หรือ zig-zig แล้วโหนดไม่ถึง root จะต้อง
ใช้ single rotation หมุนโหนดต่อ



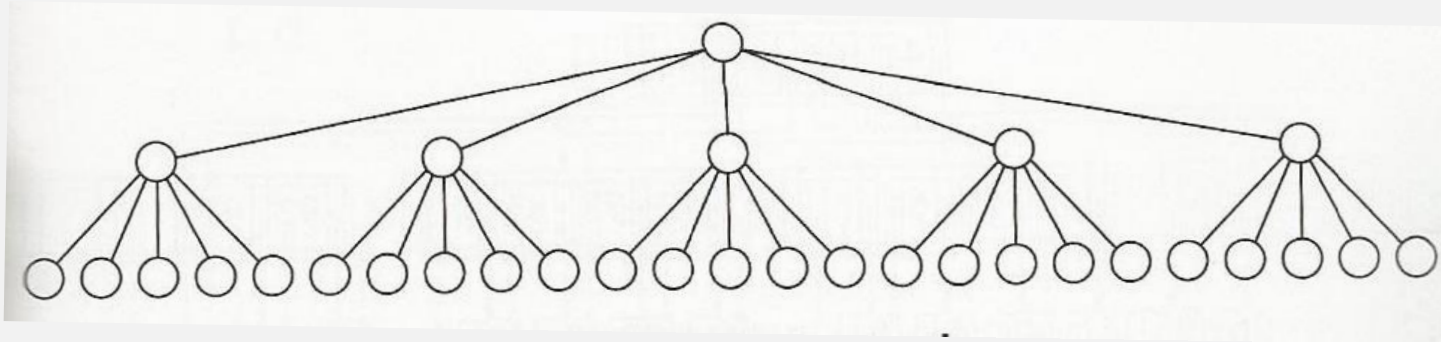
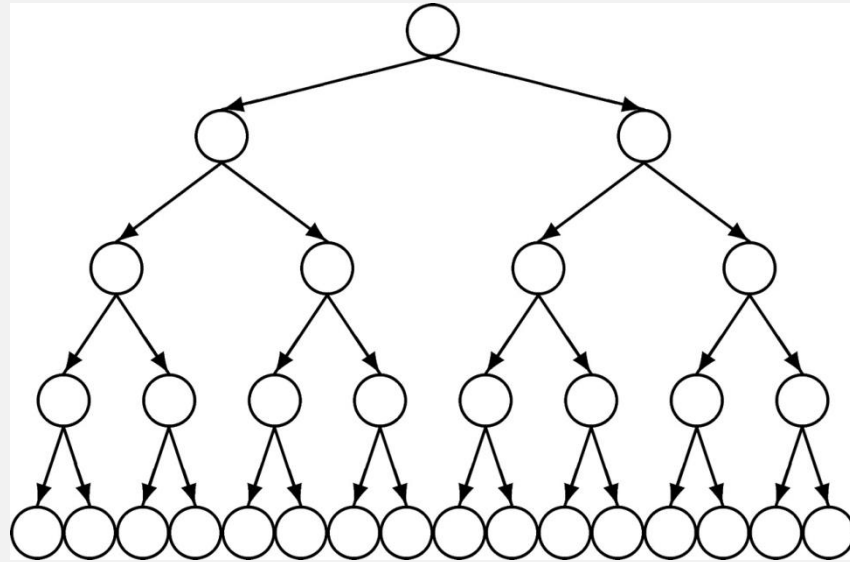




3.6 B-Trees

Complete binary

Tree 31 nodes





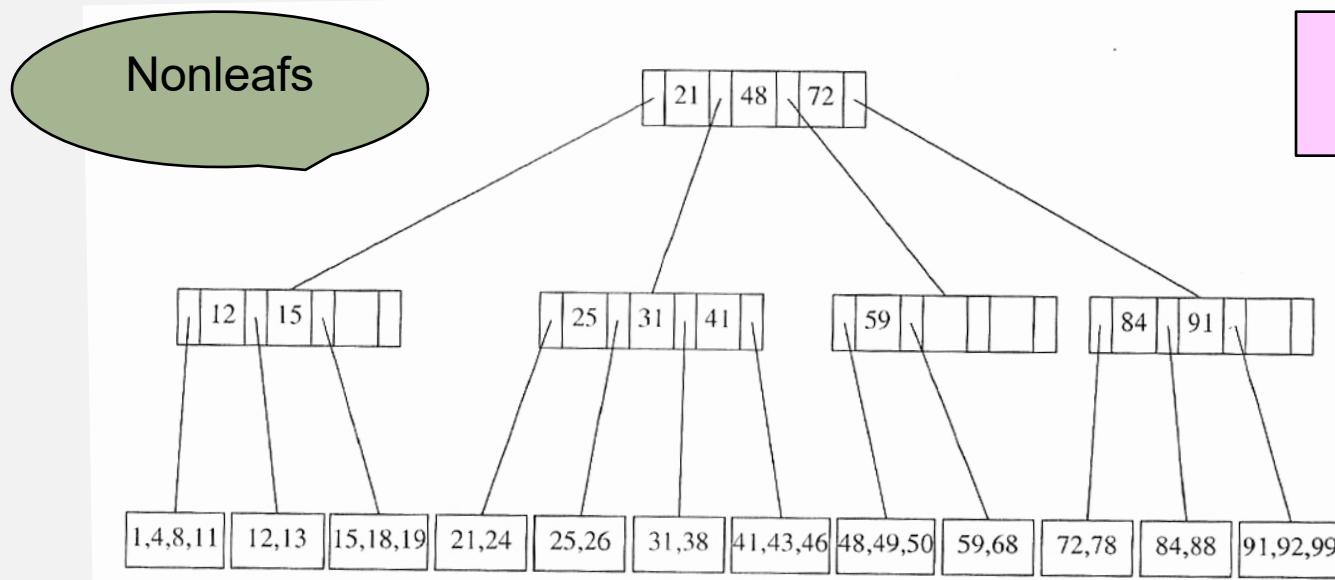
- สำหรับข้อมูลขนาดใหญ่
- ส่วนใหญ่ใช้ในระบบฐานข้อมูลและระบบไฟล์



3.6 B-Trees

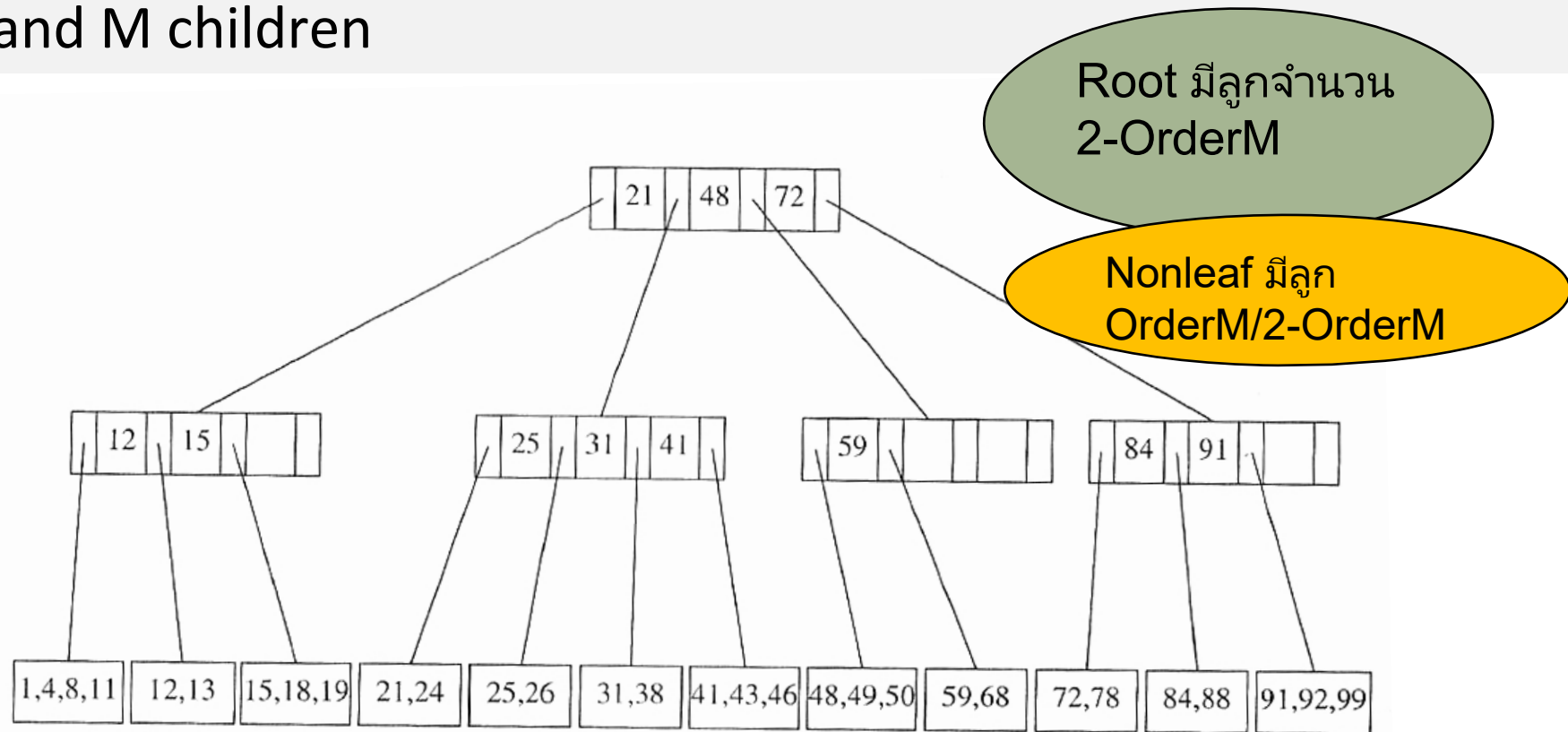
A B-tree of **order m** is a tree with the following structural properties:

- 1) The data items are stores at leaves.
- 2) The nonleafs nodes store up to $M-1$ keys to guide the searching;
key i represents the smallest key in subtrees $i+1$



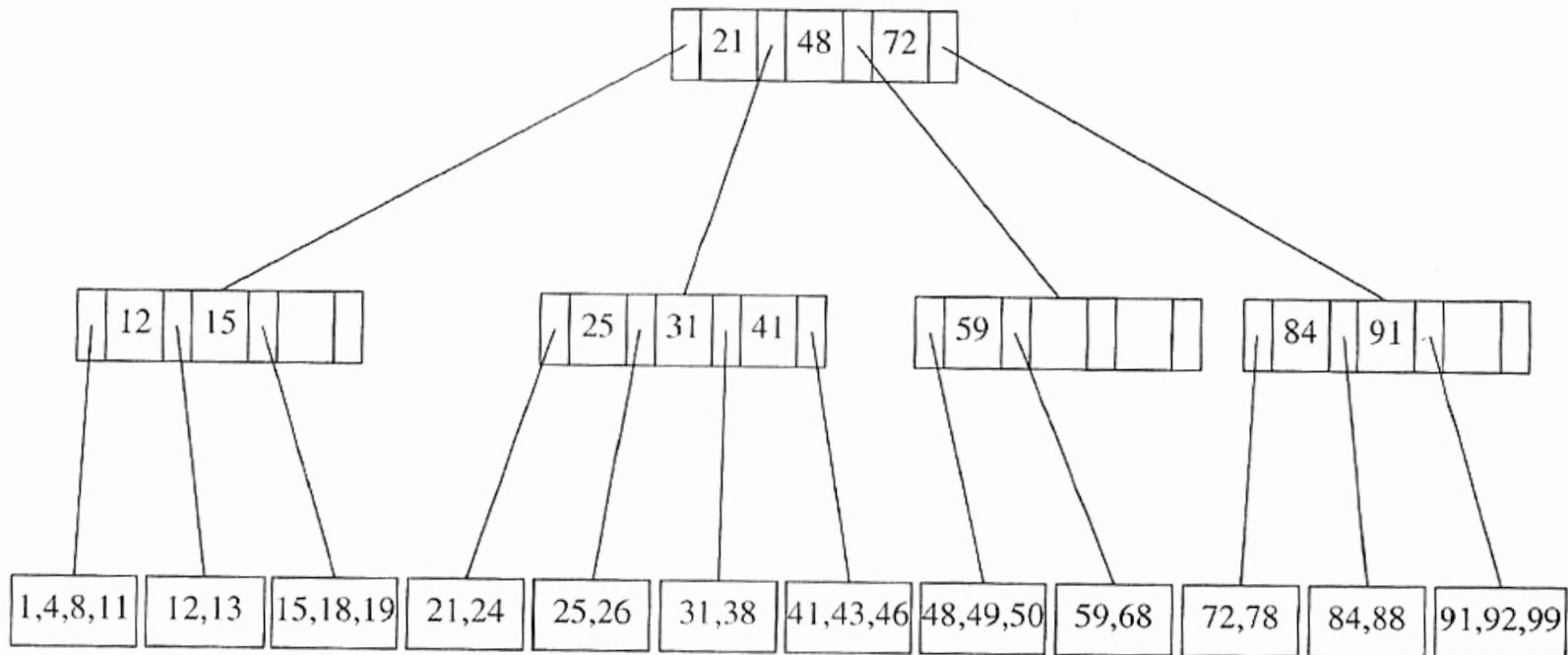


- 3) The root is either a leaf or has between two and M children.
- 4) All nonleaf nodes(Except the root) have between $\lceil M/2 \rceil$ and M children





5) All leaves are at the same depth and have between $\lceil L/2 \rceil$ and L children.



leaf มีลูกจำนวน
 $L/2$ -OrderM



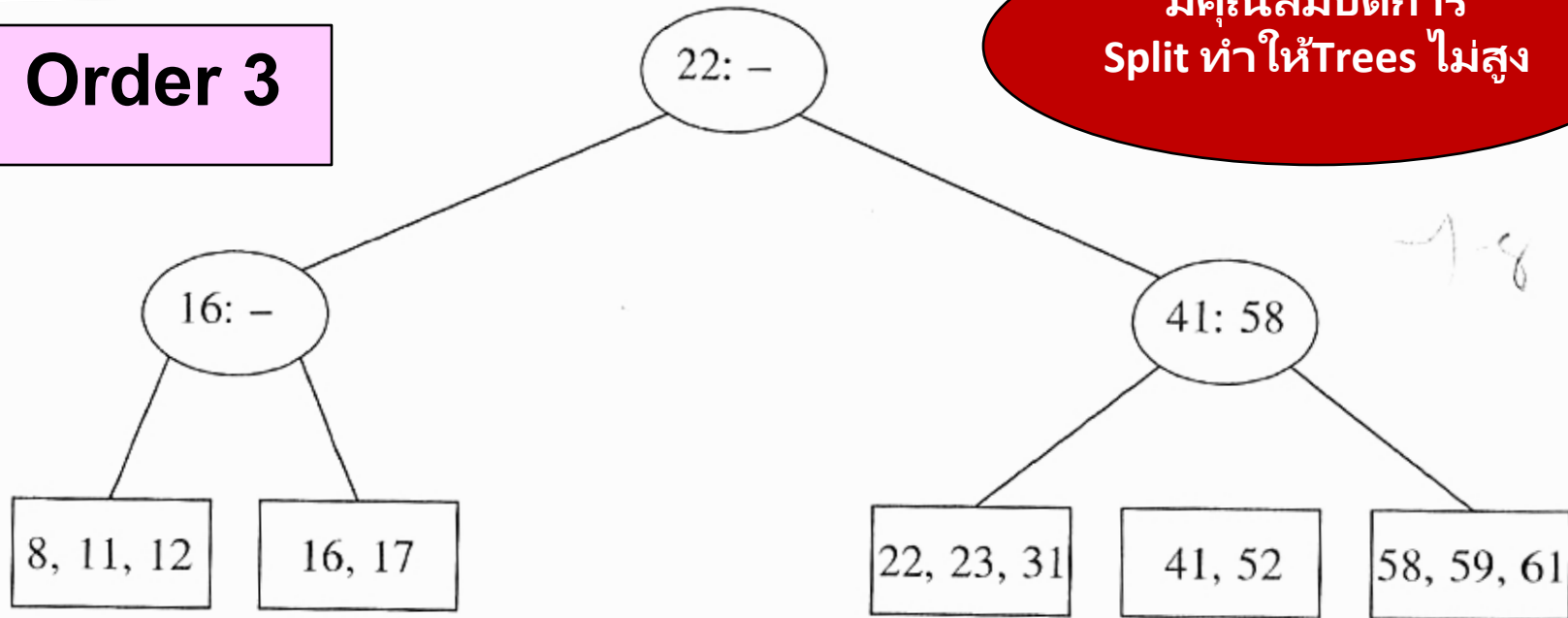
Order 3

Insert 22,16,41,58,61,59

มีคุณสมบัติการ
Split ทำให้Trees ไม่สูง



Order 3



Insert 18, 1, 19, 28



Order 5

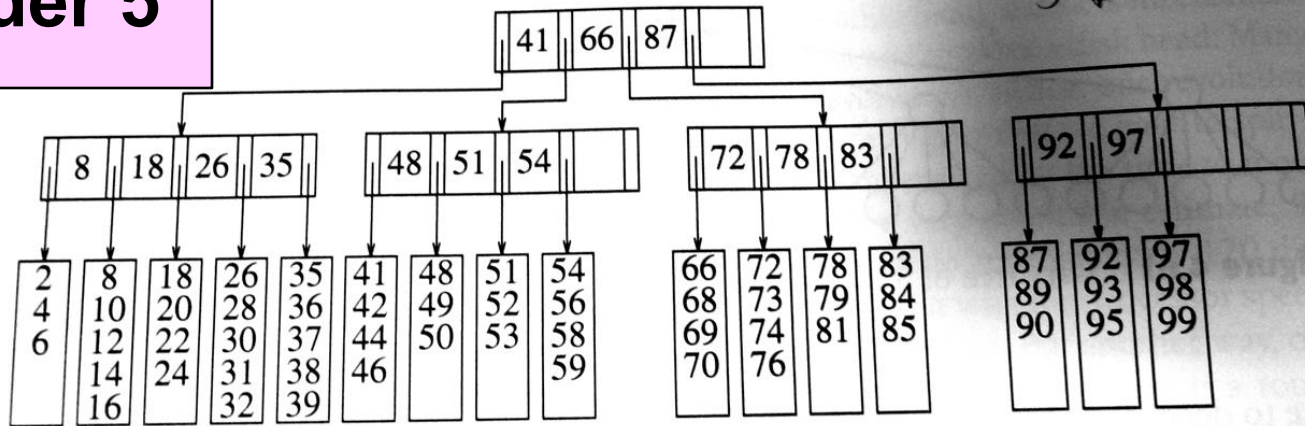
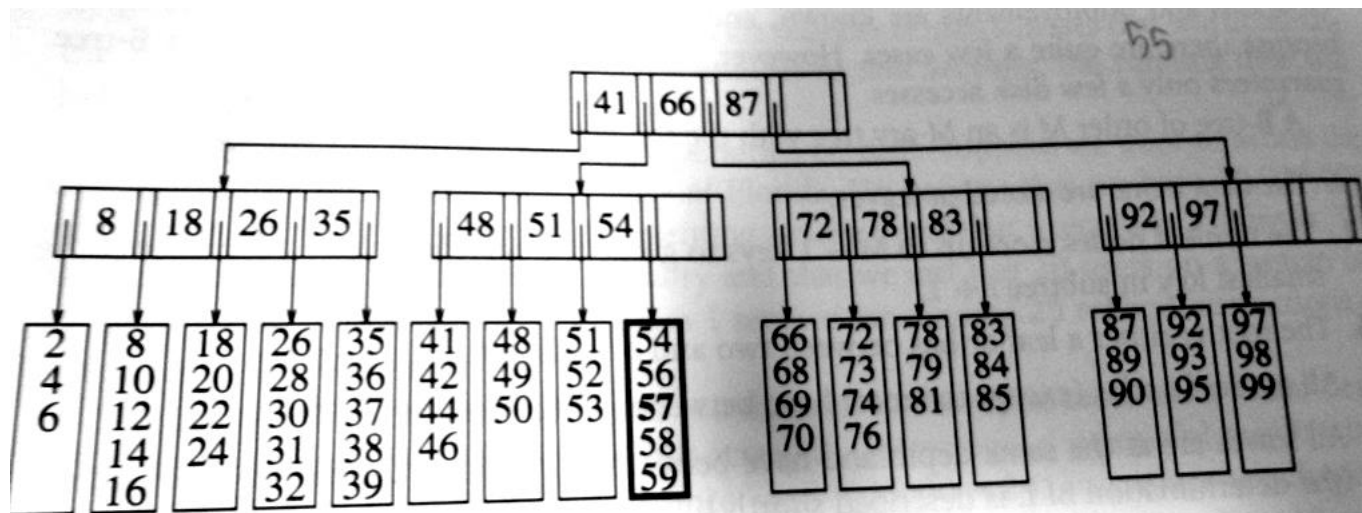
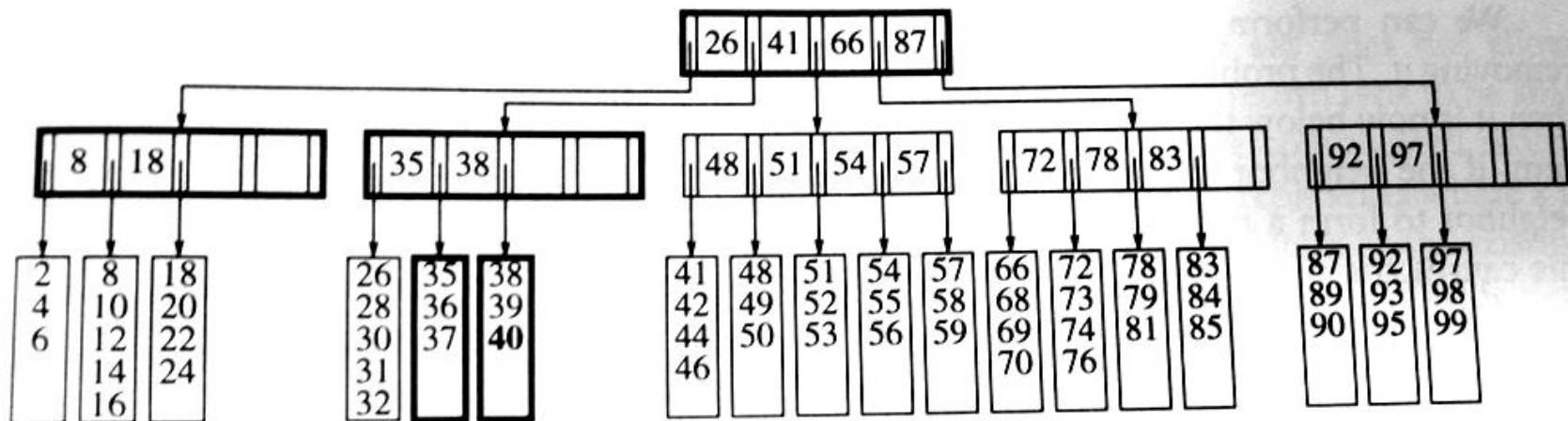
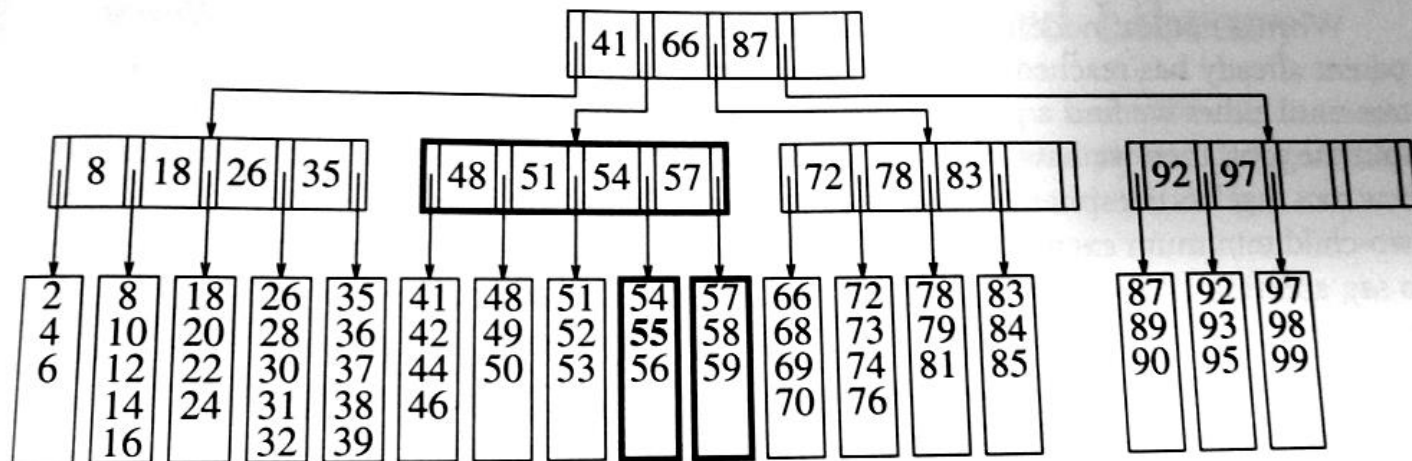


Figure 4.62 B-tree of order 5



57 is the tree in Figure 4.62





การบ้าน

1. จงเขียนโปรแกรมสร้าง expression trees เช่นถ้าต้องการ สร้าง expression trees
Input เป็นนิพจน์ postfix
 - operand คือตัวอักษรภาษาอังกฤษ 1 ตัว
 - operator ประกอบด้วย เครื่องหมาย + - * /
 - แต่ละพจน์ของ input เว้นช่องว่าง 1 space และจบ input ด้วยเครื่องหมาย .

ตัวอย่าง

Input: a b + .
Preorder: + a b
Inorder: a + b
Postorder: a b +

ตัวอย่าง

Input: a b + c d e + * + .
Preorder: + + a b * c + d e
Inorder: a + b + c * d + e
Postorder: a b + c d e + * +



2. ให้นิสิตทำการสร้าง AVL โดยกำหนด Operation ดังนี้

=====MENU=====

1) Insert

2) Print Inorder

3) Exit

Please choose >

การ insert จะทำเหมือนเดิม คือจะ insert ครั้งละ 1 ค่า จากนั้นจะกลับไปยัง เมนู

ให้ทดลอง insert ค่าดังต่อไปนี้ 10, 8, 6, 16, 14, 9, 7, 15

การพิมพ์

Inorder : 6 7 8 9 10 14 15 16